

MACHINE LEARNING FINAL PROJECT

DDoS Attack Detection Using LSTM and Autoencoder-Based Anomaly Detection

A leakage-aware ML pipeline combining supervised classification, temporal sequence learning and unsupervised anomaly detection — deployed as a live web application.

Course Code: 25SC2107E

Cybersecurity + Deep Learning

From Network Flows to Real-Time Threat Intelligence

When Traffic Becomes a Weapon

NORMAL TRAFFIC



Operates normally — requests served within expected capacity.

DISTRIBUTED MALICIOUS TRAFFIC



Overload → service degradation → unavailability for legitimate users.

- A DDoS (Distributed Denial-of-Service) attack floods a target with traffic from many sources at once.
- Legitimate requests get starved of bandwidth, connections, or compute — the service becomes unreachable.
- Modern attack traffic can closely mimic legitimate flow statistics, making simple filtering unreliable.

Why Single-Flow Rules Fall Short

- Rule-based / threshold detectors react to known signatures — they miss novel or evolving attack shapes.
- Attacks are not always visible in a single flow; they emerge as patterns across time.
- Some malicious behaviour looks statistically close to benign traffic, but deviates from the network's normal baseline.

Statistical abnormalities

Unusual feature values within a single flow

Temporal patterns

Signatures that only appear across sequences of flows

Unseen anomalies

Behaviour that doesn't match any known attack label

This project combines: **supervised classification** + **sequence learning** + **anomaly detection**

Project Objectives

Develop an end-to-end ML system capable of detecting DDoS network traffic and exposing the trained models through a usable web application.

1

Data
Preparation

2

EDA

3

Supervised
Baseline

4

Temporal
LSTM

5

Unsupervised
Anomaly Detection

6

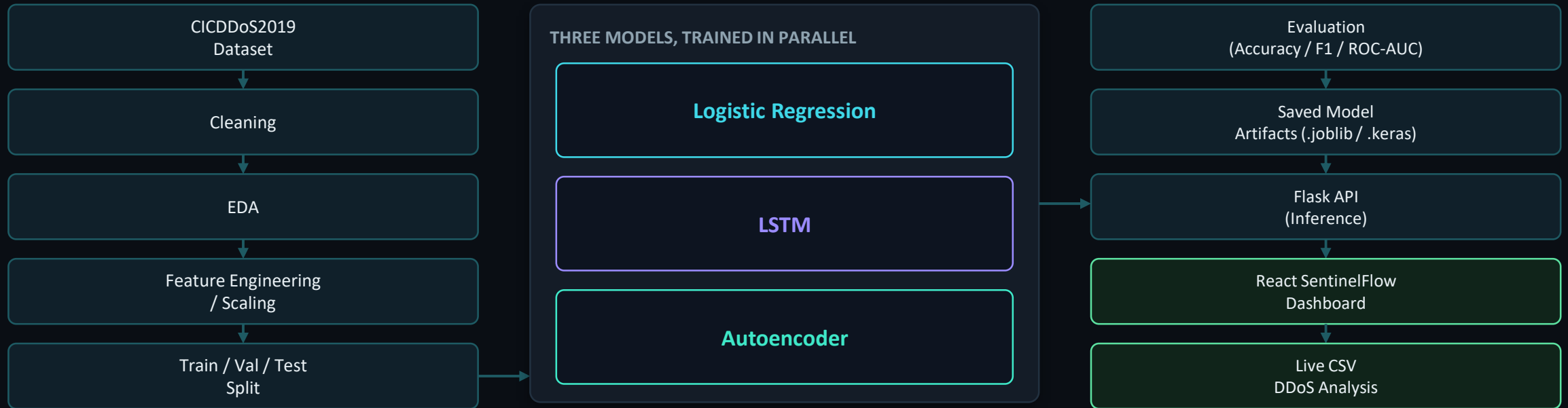
Model
Evaluation

7

Web
Deployment

The pipeline moves left to right: every stage feeds the next, ending in a deployed, user-facing system.

End-to-End System Architecture



Three complementary views of the same traffic

- Per-flow supervised classification, temporal sequence classification, and benign-only anomaly detection run side by side.
- Each is trained once and saved — the deployed website performs inference on new CSV uploads, not retraining.

The CICDDoS2019 Dataset

- A widely-used benchmark dataset of realistic, labeled DDoS network-flow traffic.
- Captures multiple distinct DDoS attack families alongside genuine BENIGN traffic.
- Each row is a network flow described by a rich set of statistical flow-level features.
- For this project, a binary detection target was derived: BENIGN vs. DDoS / Attack.

TRAFFIC CATEGORIES IN THE DATASET

BENIGN

LDAP

MSSQL

NetBIOS

Portmap

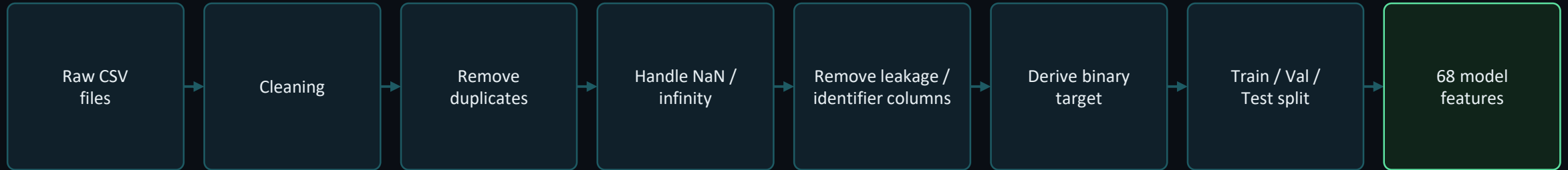
Syn

UDP

UDPLag

Curated and leakage-safe CICDDoS2019 network-flow dataset — cleaned, deduplicated and split for supervised and unsupervised modeling.

Data Preparation Pipeline



LEAKAGE PREVENTION

Flow ID, Source IP, Destination IP, raw Timestamp and SourceCSV are excluded from model input — they would let the model memorize identity rather than learn traffic behaviour. Timestamp and SourceCSV are retained only to construct ordered LSTM sequences.

68

numerical input features

42,782

training rows

9,508

validation rows

14,260

test rows

What EDA Revealed

1

BENIGN vs. DDoS distribution

Class balance was examined after cleaning to confirm the reduced dataset supports fair supervised training.

2

Attack-type distribution

Multiple DDoS families (LDAP, MSSQL, NetBIOS, Portmap, Syn, UDP, UDPLag) were inspected before collapsing to a binary target.

3

Train / Val / Test split distribution

Each split was checked to ensure BENIGN and DDoS proportions stayed consistent across partitions.

4

Feature histograms & boxplots

Flow-level statistical features were examined for scale, skew and outliers ahead of scaling.

5

Correlation analysis

Feature relationships were reviewed to understand redundancy before model input selection.

StandardScaler & Data Leakage Control

TRAIN

fit + transform

Learn μ (mean) and σ (std. dev.) from training data only**VALIDATION**

transform only

Scaled using the training-set statistics — never refit

TEST

transform only

Scaled using the training-set statistics — kept fully unseen until final evaluation

LIVE CSV (deployment)transform using saved
scaler

The exact same fitted scaler is reused for real-world inference

No test-set information is ever used to fit the scaler or select the anomaly threshold — every evaluation number reflects genuinely unseen data.

Why Three Models?

LOGISTIC REGRESSION

“Is this individual flow suspicious?”

Per-flow supervised baseline — evaluates one network flow at a time.

LSTM

“Does this sequence of flows show an attack pattern?”

Supervised temporal model — evaluates ordered 8-flow windows.

AUTOENCODER

“Does this traffic deviate from learned normal behaviour?”

Unsupervised anomaly detector — trained only on BENIGN traffic.

Logistic Regression — Supervised Baseline



```
lr = LogisticRegression(...)
lr.fit(X_train_scaled, y_train)
pred = lr.predict(X_test_scaled)
```

A traditional supervised classifier — the simple, interpretable baseline every advanced model must beat.

FINAL TEST METRICS

99.7826%

Accuracy

99.8876%

Precision

99.6774%

Recall

99.7824%

F1-score

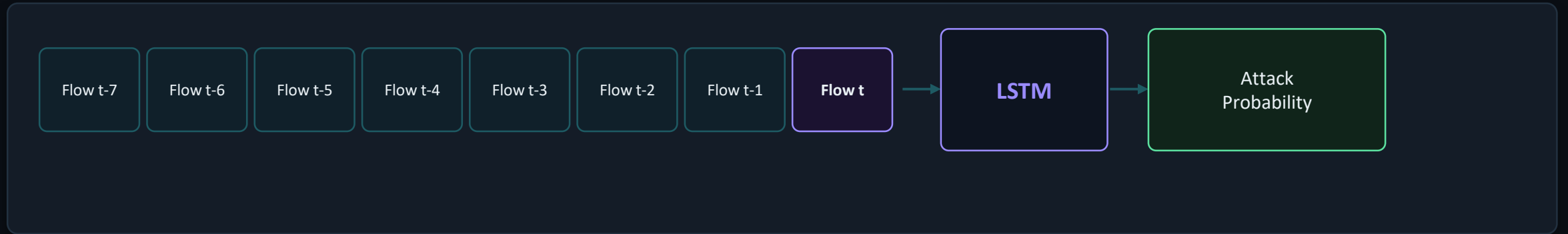
99.9270%

ROC-AUC

99.9489%

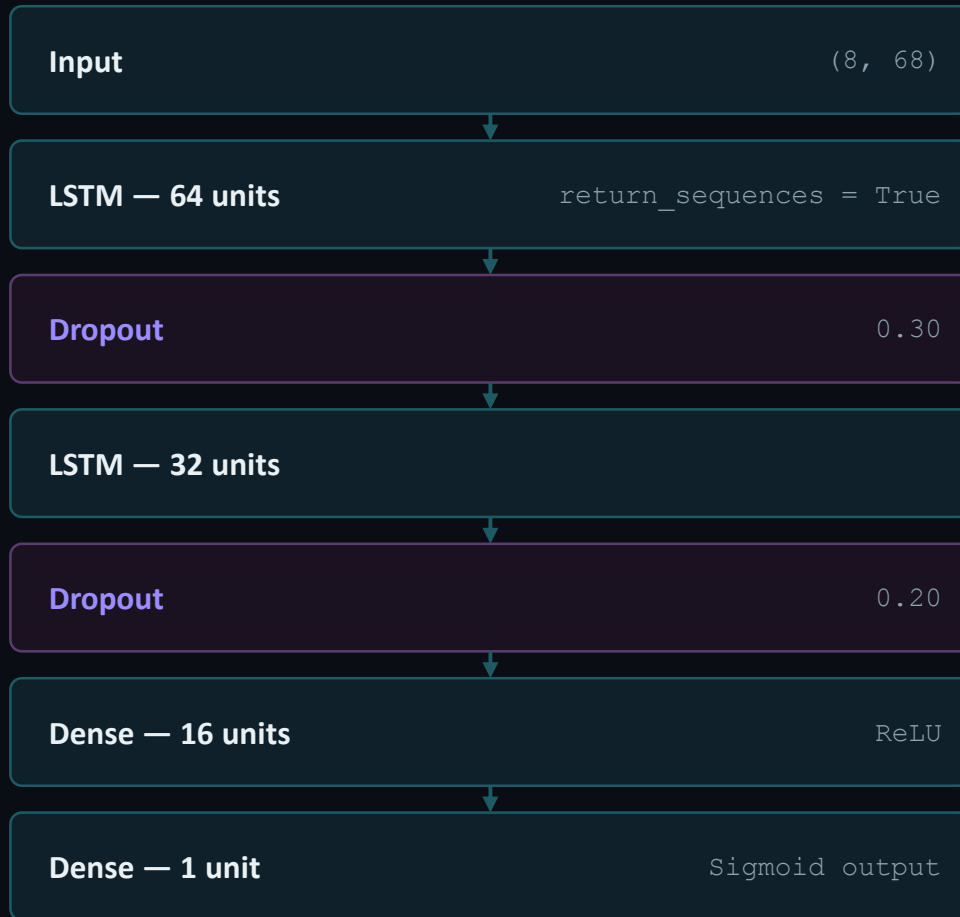
PR-AUC

LSTM — Temporal Sequence Detection



- Sequences are grouped by SourceCSV so a window never crosses unrelated traffic captures.
- Flows are ordered chronologically by Timestamp before windowing.
- Each window covers 8 consecutive flows (68 features each) — the label is the class of the final flow in the window.
- This lets the model learn how flow statistics evolve leading up to an attack, not just a single snapshot.

LSTM Network Architecture



```
Sequential([
    Input(shape=(8, 68)),
    LSTM(64, return_sequences=True),
    Dropout(0.30),
    LSTM(32),
    Dropout(0.20),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])
```

47,009

trainable parameters

Adam optimizer • Binary Cross-Entropy
Batch size 256 • up to 15 epochs
EarlyStopping + ModelCheckpoint

LSTM Training & Final Results

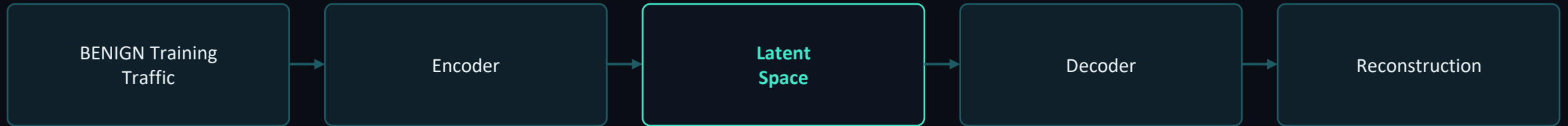
TRAINING BEHAVIOUR

- Training / validation accuracy tracked across epochs to monitor fit.
- Training / validation loss (Binary Cross-Entropy) confirmed convergence without divergence.
- EarlyStopping halted training once validation loss stopped improving, preventing overfitting.
- ModelCheckpoint preserved the best-performing weights on the validation set.
- Confusion matrix, ROC curve and PR curve were generated on the held-out test set.

FINAL TEST METRICS

99.8593%	99.9294%
Accuracy	Precision
99.7886%	99.8590%
Recall	F1-score
99.9902%	99.9916%
ROC-AUC	PR-AUC

Autoencoder — Learning What's Normal



NORMAL TRAFFIC

Original $\approx$ Reconstruction

→ *small reconstruction error*

The Autoencoder has seen traffic like this many times during training and reconstructs it accurately.

ATTACK TRAFFIC

Original $\neq$ Reconstruction

→ *high error* → *flagged as anomaly*

Because it was never trained on attack behaviour, the model struggles to reconstruct it faithfully — that gap is the signal.

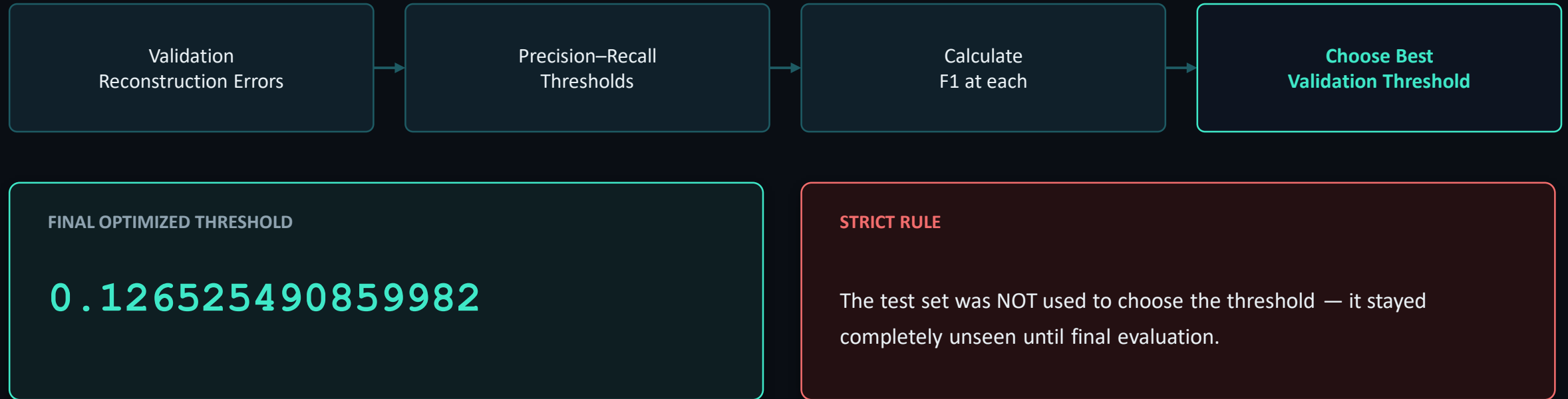
Autoencoder Architecture



68 → 48 → 24 → 12 → 24 → 48 → 68

- The 12-dimensional latent bottleneck forces the network to learn a compressed representation of normal traffic.
- Trained exclusively on BENIGN samples from the training split ($X_{\text{train_benign}} = X_{\text{train_scaled}}[y_{\text{train}} == 0]$) — it never sees attack labels.
- Its objective is reconstruction fidelity on normal behaviour, not classification.

Anomaly Threshold Selection



- Reconstruction error above the threshold → flagged as DDoS / anomalous.
- Selecting the threshold on validation data (not test) keeps the final test metrics an honest, unbiased estimate of real-world performance.

Autoencoder Final Results

TEST METRICS

90.1262%

Accuracy

83.7303%

Precision

99.6073%

Recall

90.9813%

F1-score

91.4257%

ROC-AUC

86.0693%

PR-AUC

CONFUSION MATRIX

5750

TN

1380

FP

28

FN

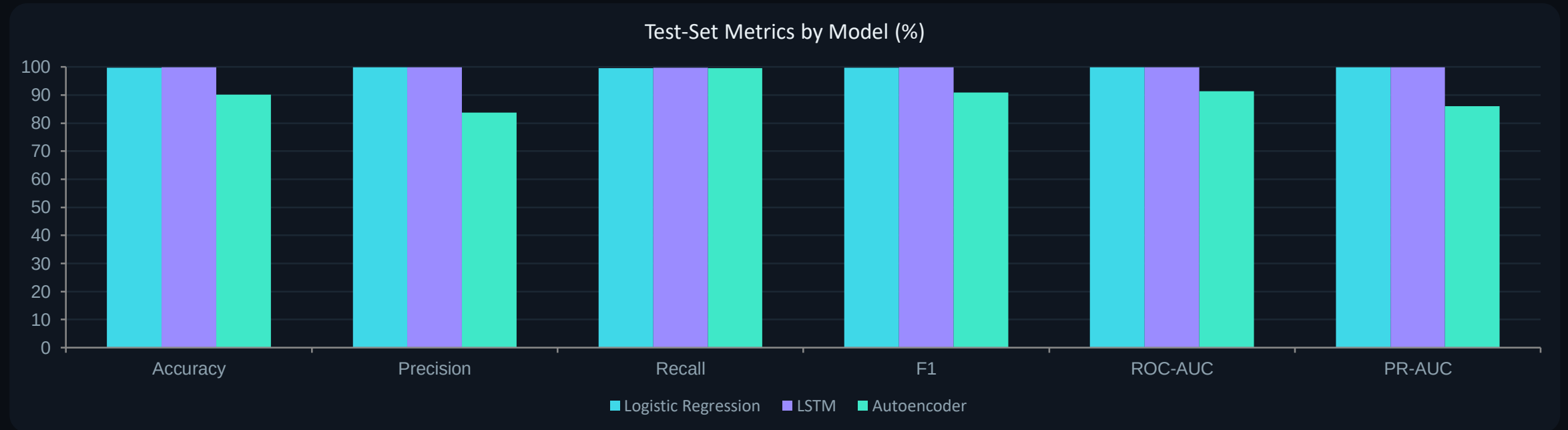
7102

TP

Only 28 DDoS samples were missed out of the entire test evaluation.

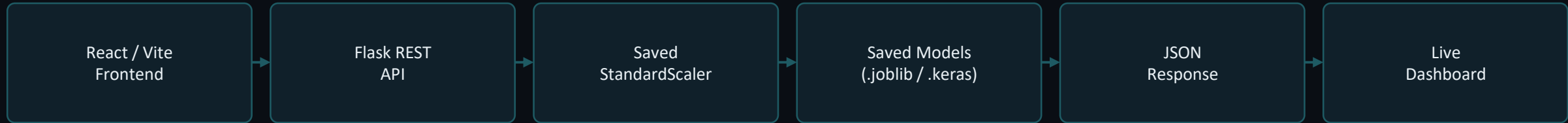
- Lower precision and overall accuracy than the supervised models, but extremely high attack recall (99.6%).
- This demonstrates its value as a high-sensitivity anomaly detector — it generates more false positives in exchange for rarely missing a real attack.
- The Autoencoder did not “fail” — it trades some precision for sensitivity, which is a deliberate and useful property in a detection system.

Final Model Comparison



- Logistic Regression — supervised, individual-flow baseline. LSTM — supervised, temporal sequence detector.
- Autoencoder — benign-only, unsupervised anomaly detector. Its lower accuracy reflects a different (harder) task, not a weaker model.

SentinelFlow Deployment Architecture



TECH STACK

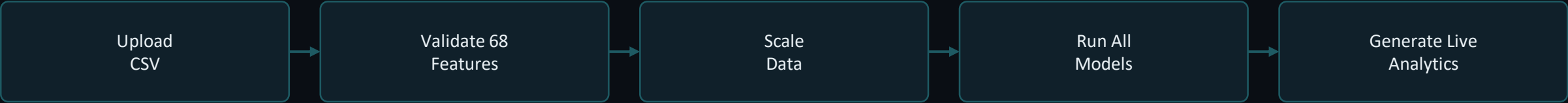
- Frontend: React + Vite
- Backend: Python + Flask
- ML: TensorFlow / Keras, scikit-learn, Pandas, NumPy, Joblib

SAVED ARTIFACTS

```
logistic_regression.joblib • lstm_model.keras •  
autoencoder_model.keras • scaler.joblib •  
feature_columns.json • autoencoder_threshold.json •  
metrics/config JSON
```

The website performs inference only. Models are trained once, saved, and reused — a new CSV upload never retrains them.

Live CSV DDoS Analysis



sentinelflow.app / dashboard

Total flows analyzed	BENIGN prediction count	DDoS / suspicious count	DDoS percentage / risk
LR prediction stats	LSTM attack probabilities	Autoencoder recon. errors	Autoencoder anomaly count
Model attack-rate comparison	Model agreement	BENIGN vs DDoS distribution	Recon-error histogram

Fixed Research Results vs. Live Results

RESEARCH RESULTS — FIXED

- Accuracy, Precision, Recall, F1
- ROC-AUC, PR-AUC
- Original confusion matrices
- Training loss / validation loss
- Training / validation accuracy
- ROC curves, PR curves

Calculated once during the controlled training / testing experiment.

LIVE CSV ANALYTICS — DYNAMIC

- BENIGN vs. DDoS distribution
- Suspicious traffic percentage
- Model attack rates
- Autoencoder anomaly score distribution
- Reconstruction error
- Model agreement & risk level

Recomputed every time a new CSV is uploaded and analyzed.

Key Contributions

- 1 Leakage-aware preprocessing pipeline
- 2 Conventional supervised baseline (Logistic Regression)
- 3 Real 8-flow temporal LSTM sequence windows
- 4 Benign-only Autoencoder training
- 5 Validation-only anomaly threshold selection
- 6 Multiple evaluation metrics per model
- 7 Model serialization for reuse (.joblib / .keras)
- 8 Working Flask inference deployment
- 9 React SentinelFlow dashboard
- 10 Live, dynamic CSV analytics

Conclusion & Future Scope

CONCLUSION

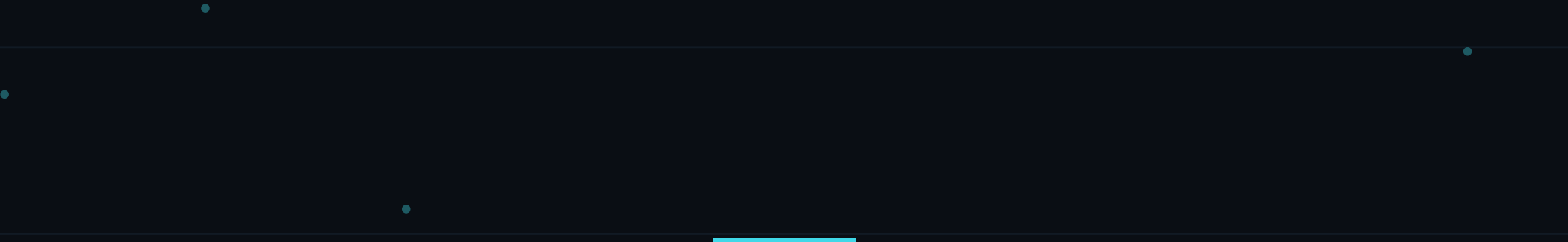
The system demonstrates a complete ML lifecycle from cybersecurity data preparation through model training, anomaly detection, evaluation and deployment.

FUTURE SCOPE

- Live packet capture / flow generation
- Online network monitoring
- Streaming inference
- Adaptive anomaly thresholds
- Attack-family classification
- Model drift monitoring
- Explainable AI
- Deployment on cloud / SOC infrastructure

Not yet implemented — proposed directions for extending this work.

“From Network Flows to Real-Time Threat Intelligence”

A decorative graphic consisting of several small teal dots scattered across the upper half of the slide, and a short teal horizontal line centered above the 'THANK YOU' text.

THANK YOU

Questions?